

NEWS APPLICATION – NEWS SNAP

Submitted By

Vijay Reddy | AP24110010229

Keerthi Sai V | AP24110010238

Koushik Ch | AP24110010250

Rupa Sri J | AP24110010274

Sandhya Rani K | AP24110010278



Academic Year 2025–2026

Technology Stack: [React.js](#) | [Node.js](#) | [Express.js](#) | [MongoDB](#)

INTRODUCTION

The News Application is a comprehensive full-stack web platform built using the MERN stack — MongoDB, Express.js, React.js, and Node.js. It is designed to deliver a seamless news browsing, publishing, and management experience to readers, content administrators, and casual visitors alike.

The platform aggregates live India headlines from the NewsData.io API when configured, and gracefully falls back to a MongoDB-persisted article store or bundled sample data, ensuring continuous availability at all times. With a clean, category-driven interface, users can browse top stories, search by keyword, filter by topic, and deep-read individual articles. Authenticated users enjoy personalised features such as a Watch Later saved-articles list, while administrators can publish, edit, and remove stories directly from the platform.

The application follows a well-defined MVC architecture on the server and a component-based React architecture on the client. JWT-based authentication with bcrypt-hashed passwords guarantees security, and a dedicated SMTP-driven forgot-password flow makes account recovery straightforward. The project demonstrates best practices in REST API design, Mongoose schema modelling, React context, protected routing, and responsive UI composition.

SCENARIO BASED CASE STUDY

Meet Priya, a busy journalism student who wants a single platform where she can read curated news, save articles for later review, and occasionally publish opinion pieces for her college blog. Priya discovers the News Application — a full-stack web platform tailored for news consumption and publishing. She explores the platform and finds it perfectly aligned with her daily workflow:

User Registration and Authentication:

Priya registers an account by providing her name, e-mail, gender, and a strong password. She logs in with JWT-secured credentials, gaining access to personalised features while her data remains private and encrypted.

News Feed and Category Browsing:

On the home page Priya sees a curated feed of featured headlines alongside category tabs — World, Politics, Technology, Science, Health, Sports, Entertainment, Culture, and more. She switches between tabs to find stories relevant to her coursework.

Search and Filtering:

When researching for an assignment, Priya uses the search bar to look up articles by keyword. She can also filter by tag or category, narrowing thousands of articles down to exactly what she needs.

Reading Articles:

Clicking any card takes Priya to a full Article Detail page that shows the headline, author, publish date, cover image, and the complete story body. Estimated read-time badges help her decide whether she has time to read on the spot or should save it.

Watch Later (Saved Articles):

Priya bookmarks articles she cannot read immediately with the Save button. All saved stories appear on her dedicated Saved page, persisted in MongoDB and tied to her account.

Admin Publishing:

Her college journalism professor, logged in as an Admin using the secret access code, publishes new campus news stories through the Publish page — entering a title, excerpt, content, category, tags, cover image URL, and read time. The story appears on the feed instantly.

Password Recovery:

When Priya forgets her password, she uses the Forgot Password flow. The system generates a secure time-limited reset link, e-mails it via SMTP (or logs it locally during development), and Priya resets her password within the hour.

Thanks to the News Application, Priya manages her reading list efficiently, stays informed, and collaborates with her professor to publish content — all within a single, secure platform.

SYSTEM REQUIREMENTS

To ensure smooth development, deployment, and usage of the News Application, certain system prerequisites must be met. These requirements are categorised into software, hardware, and setup specifications.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- Operating System: Windows 10/11, macOS, or Linux — supports cross-platform development and testing.
- Node.js (v16 or above): Provides the runtime environment for the Express backend and Vite-based React frontend.
- npm (v8 or above): Package manager used to install all server and client dependencies.
- React.js (v18): JavaScript library for building the dynamic, responsive news feed interface.
- Vite: Lightning-fast build tool and development server for the React client.
- Express.js: Lightweight framework for building the RESTful JSON API.
- MongoDB (Atlas or local): NoSQL database for persisting users, articles, and saved-article records.
- Mongoose: Schema-based ODM used to model and validate all MongoDB documents.
- NewsData.io API Key: External news API that supplies live India headlines when configured.
- Nodemailer + SMTP Credentials: Required to deliver password-reset e-mails in production.
- Postman: API testing tool used during backend development.
- Visual Studio Code: Preferred code editor with Git and integrated terminal support.
- Google Chrome / Firefox (latest): For rendering and testing the UI in real time.

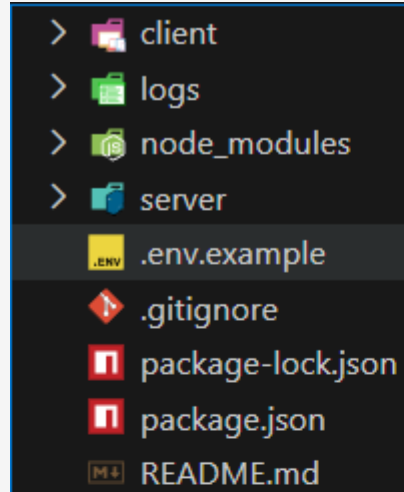
2. Hardware Requirements

Minimum and recommended specifications needed to support development and usage of the application.

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — ensures fast compilation and multitasking during development.
- RAM: Minimum 8 GB (16 GB recommended) — for handling the development servers, IDE, and browser testing simultaneously.
- Storage: At least 1 GB free space — required for package installations, MongoDB setup, and local project files.
- Display: 1366x768 or higher — recommended for an optimal coding and application layout experience.

- Internet Connection: Required to fetch live news from NewsData.io and to connect to MongoDB Atlas.

PROJECT ARCHITECTURE



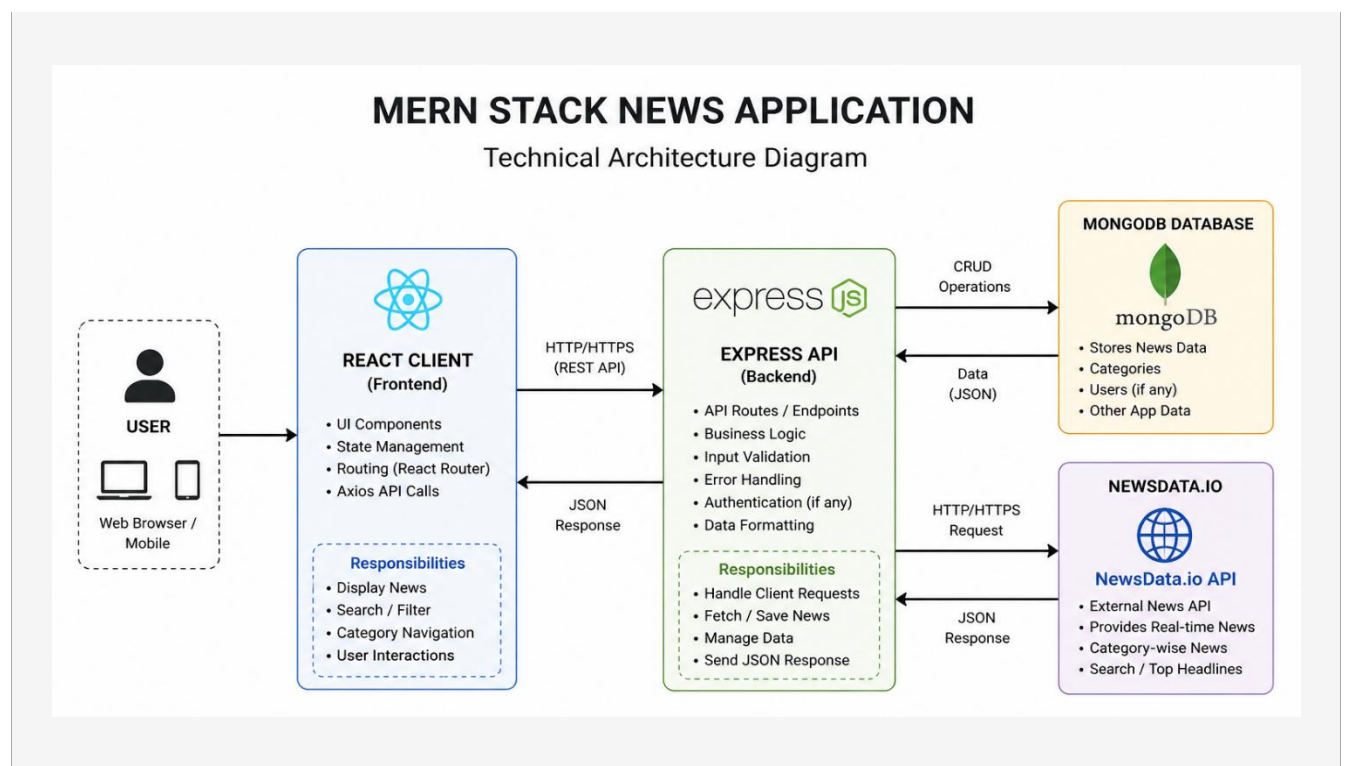
A screenshot of a file explorer window with a dark theme. The file list is as follows:

- > client (folder icon)
- > logs (folder icon)
- > node_modules (folder icon)
- > server (folder icon)
- .env.example (file icon)
- .gitignore (file icon)
- package-lock.json (file icon)
- package.json (file icon)
- README.md (file icon)

The application follows a clear separation between the client and server. Both are maintained as independent npm workspaces inside a single monorepo root. A concurrently script in the root package.json starts both the React Vite dev server and the Express API simultaneously with a single npm run dev command.

The client is a React single-page application bootstrapped with Vite. All user-facing pages, components, contexts, and API service calls live under client/src/. The server is a pure REST API built with Express and Mongoose, organised into routes, controllers, models, middleware, services, and configuration — mirroring the MVC pattern described later in this document.

TECHNICAL ARCHITECTURE



The data flow begins with the user interacting with the React frontend running in the browser. Every user action that requires remote data — loading the article feed, searching, logging in, saving an article — triggers an Axios HTTP call to the Express REST API.

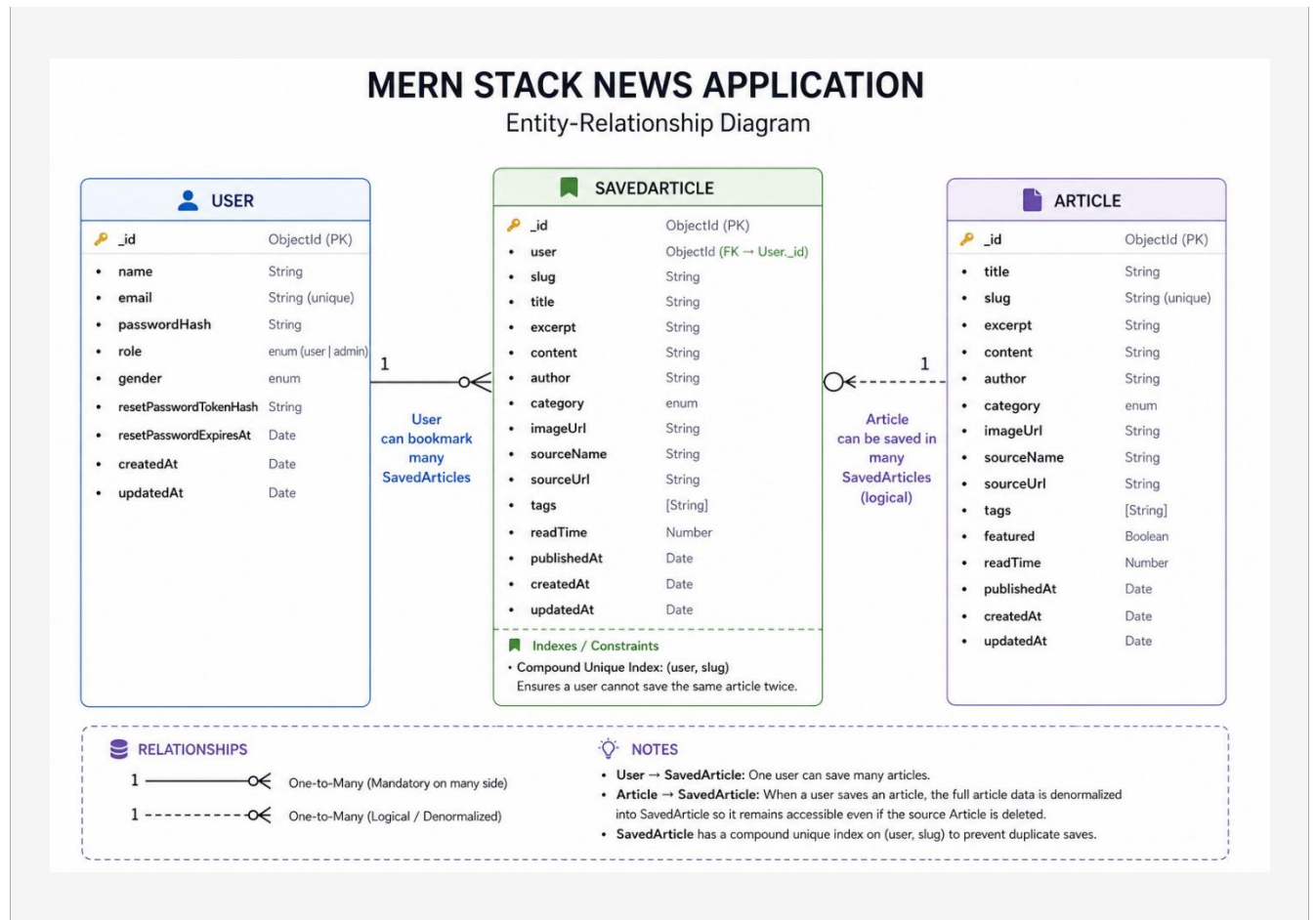
The Express API applies CORS, Helmet security headers, JWT authentication middleware (where required), and request-body parsing before passing control to the appropriate controller. The controller decides which data source to consult:

- If a valid `NEWS_API_KEY` environment variable is set, the `newsApiService` fetches live India headlines from NewsData.io and returns them directly without writing to the database.
- If NewsData.io is unavailable or unconfigured, the controller queries MongoDB through Mongoose models.

- If MongoDB is also unavailable, the controller serves a bundled set of sample articles from an in-memory local store.

For write operations (create, update, delete articles; register/login users; save articles; password reset), the controller always persists to MongoDB. Responses flow back as JSON, and the React client updates its UI state accordingly. File uploads (e.g. cover image URLs) are currently handled as URL strings rather than binary blobs, keeping the storage layer simple.

ER DIAGRAM



The Entity-Relationship diagram for the News Application represents the entities and their relationships within the data model. Below is a description of each entity and its attributes.

Primary Keys

Each MongoDB collection uses the default ObjectId `_id` field as its primary key, automatically generated and globally unique.

Foreign Keys

The SavedArticle collection stores a user field of type ObjectId with a ref to the User model, acting as a foreign key that links saved records to their owner. An index on this field ensures fast look-ups of all saves belonging to a given user.

KEY FEATURES

The News Application provides a rich set of features for readers, administrators, and developers. The table below summarises the core capabilities of the platform.

Feature	Description
Live News Feed	Fetches real-time India headlines from NewsData.io when an API key is configured, with automatic fallback to MongoDB and sample data.
Category Tabs	Ten article categories — World, Politics, Business, Technology, Science, Health, Sports, Entertainment, Culture, Local — with a single-click filter.
Full-Text Search	Keyword-based search across article titles, excerpts, content, and tags using MongoDB text indexes and client-side filtering for sample data.
Article Detail Pages	Slug-based SEO-friendly URLs for each article, showing full content, cover image, author, publish date, and read-time estimate.
User Authentication	JWT-based register and login with bcrypt-hashed passwords. Role-based access control separates regular users from administrators.
Admin Publishing	Admins can create, edit, and delete articles through a protected Publish page. Admin registration and login require a secret access code.
Watch Later (Saved Articles)	Authenticated users can save articles to a personal list, persisted in MongoDB and accessible on a dedicated Saved page.
Forgot / Reset Password	Secure SHA-256-hashed reset tokens with 1-hour expiry, delivered via Nodemailer SMTP. Falls back to returning the link in the API response during local development.
Gender-Aware Avatar Badges	Profile avatars adapt to the user's selected gender identity using the AvatarBadge component.
Responsive UI	Clean responsive layout built with plain CSS, optimised for desktop and tablet viewports.
API Health Endpoint	GET /api/health reports live database connectivity status, useful for deployment monitoring.
Seed Script	A Node.js seed script populates MongoDB with sample articles for quick local development setup.

ROLES AND RESPONSIBILITIES

User (Reader)

- **Registration and Account Management:** Readers create an account by supplying their name, e-mail, gender, and password. They are responsible for maintaining their credentials and updating profile information.
- **News Browsing:** Readers can freely browse the article feed, apply category filters, and use the search bar without requiring authentication.
- **Saving Articles:** Authenticated readers can save articles to their Watch Later list and manage that list from the Saved page.
- **Password Recovery:** Readers who forget their password can initiate a self-service reset via the Forgot Password page.
- **Communication:** Readers may use links provided in articles (sourceUrl) to visit original sources and may provide feedback through external channels.

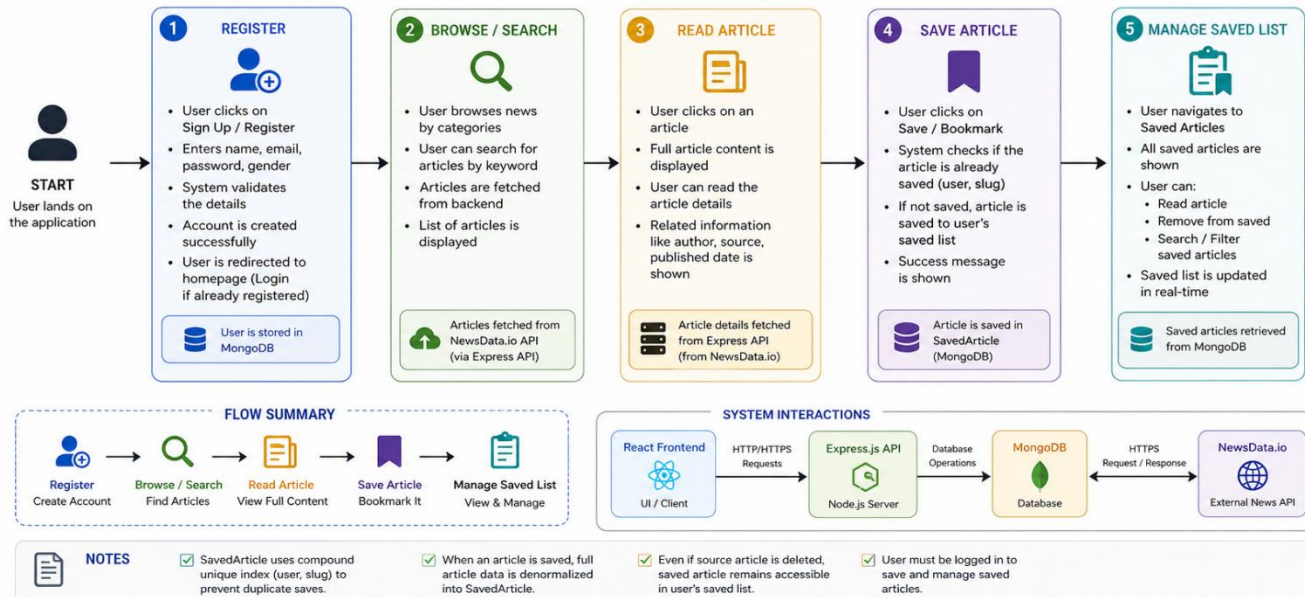
Admin

- **System Management:** The admin is responsible for overall content governance — monitoring the article feed, managing published stories, and ensuring data quality.
- **Article Publishing:** Admins create new articles through the Publish page, supplying the title, excerpt, body, category, tags, cover image URL, and read time. Admin identity is verified using a secret access code at registration and login.
- **Article Editing and Deletion:** Admins can update existing articles (PUT /api/articles/:slug) or remove outdated content (DELETE /api/articles/:slug) via the API or through any connected admin UI.
- **User Oversight:** Admins have API-level visibility into registered users and can enforce data integrity across the platform.

USER FLOW

MERN STACK NEWS APPLICATION

Customer Flow Diagram

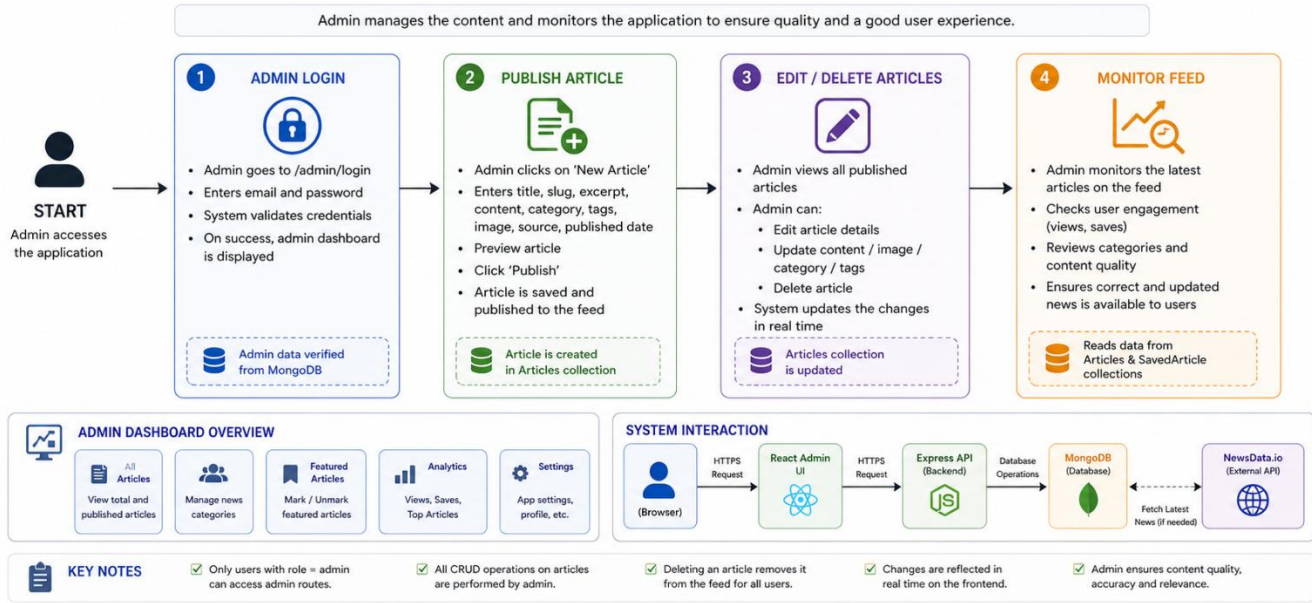


Reader Flow

- Registration and Account:** The reader visits `/auth`, selects the Register tab, fills in name, e-mail, gender, and password, and submits to create an account. A JWT token is returned and stored in the browser.
 - The reader can also log in on the Login tab if they already have an account.
- Browse and Search:** On the home page the reader scrolls the featured feed, clicks category tabs, or types in the search bar. The client debounces the query and calls `GET /api/articles` with the appropriate filters.
- Read Article:** Clicking a card navigates to `/article/:slug` where the full story is rendered.
- Save Article:** The reader clicks the Save button on a detail page. A `POST /api/saved-articles` request persists the article. The button changes to reflect the saved state.
- Manage Saved List:** On the `/saved` page the reader views all bookmarks and can delete any entry.
- Password Recovery:** On `/forgot-password` the reader enters their e-mail. If the account exists, a reset link is sent. Following the link navigates to `/reset-password/:token` where a new password is set.

MERN STACK NEWS APPLICATION

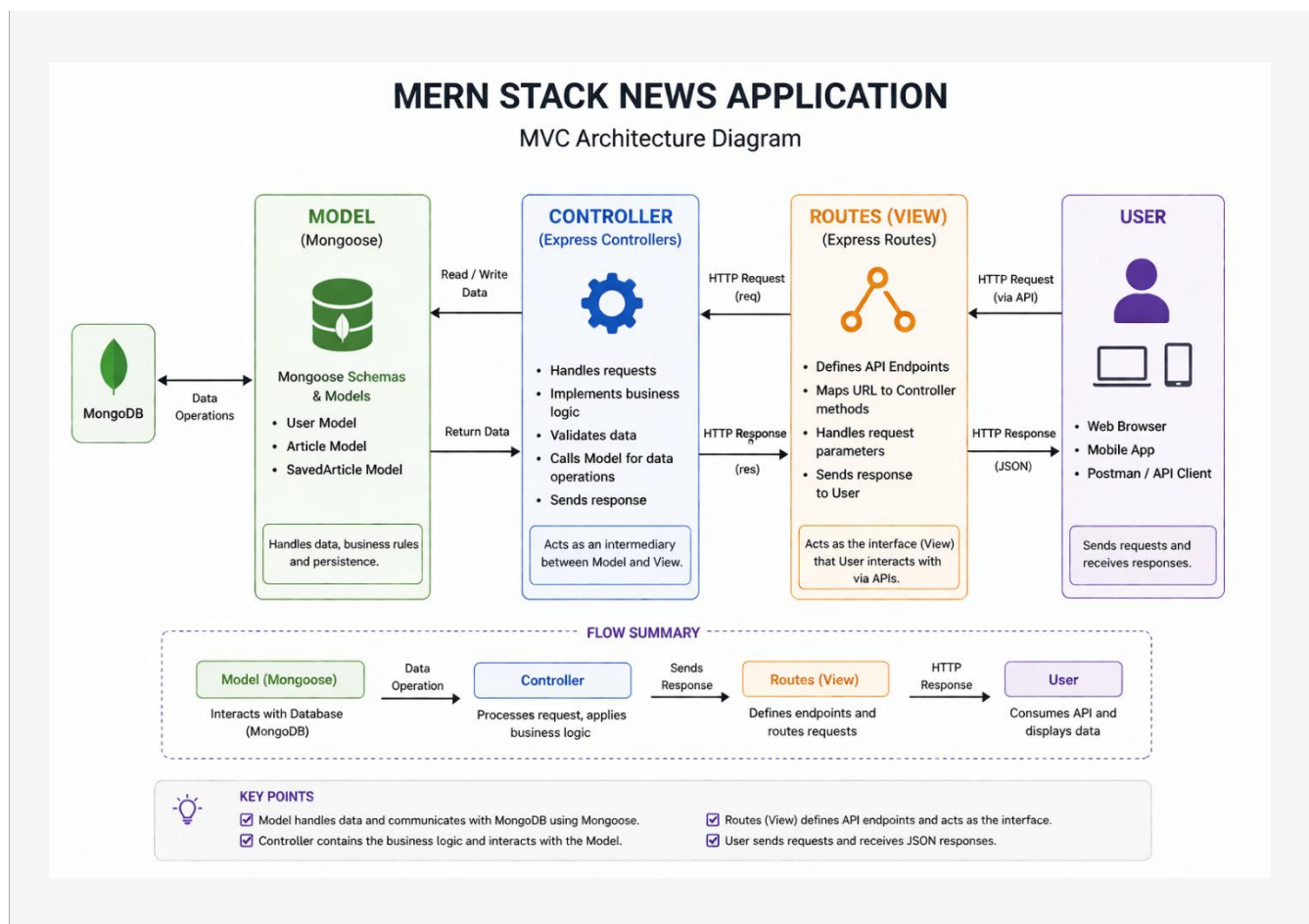
Admin Flow Diagram



Admin Flow

- **Admin Login:** The admin navigates to /auth, selects the Admin Login tab, enters credentials and the secret access code, and receives a JWT with role: admin.
- **Publish Article:** The admin visits /publish (protected, admin-only route), fills in all article fields, and submits. POST /api/articles creates the record in MongoDB.
- **Edit / Delete:** The admin calls PUT or DELETE on /api/articles/:slug to update or remove stories.

MVC PATTERN



The News Application's server-side code strictly follows the Model-View-Controller (MVC) architectural pattern. This separation of concerns improves readability, testability, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for all data-related logic. Schemas are defined with Mongoose, which provides schema validation, type casting, pre-save hooks (e.g. automatic slug generation), and text indexes. The three core models are:

- Article.js — Defines the article schema with fields for title, slug (auto-generated from title), excerpt, content, author, category, imageUrl, tags, featured, readTime, and publishedAt. Includes a pre-validate hook that slugifies the title, and compound text/category-date indexes for efficient querying.
- User.js — Defines the user schema with e-mail (unique, lowercase), passwordHash, role (user | admin), gender, and password-reset token fields. Provides comparePassword() and toSafeJSON() instance methods.

- SavedArticle.js — Defines the saved-article schema linking a User ObjectId to a denormalised article payload with a compound unique index on (user, slug).

Controller Layer

Controllers act as intermediaries between routes and models. They receive incoming requests, validate and transform input, call the appropriate model or service methods, and return structured JSON responses.

- articleController.js — Implements getArticles, getArticleBySlug, createArticle, updateArticle, and deleteArticle. Intelligently selects between NewsData.io, MongoDB, and the local in-memory fallback based on availability.
- authController.js — Implements register, login, getCurrentUser, forgotPassword, and resetPassword. Handles role validation, admin-access-code checks, bcrypt hashing, JWT creation, and SHA-256 reset-token management.
- savedArticleController.js — Implements getSavedArticles, saveArticle, and removeSavedArticle for the authenticated user's watch list.

View Layer (Routing Layer)

In the context of the backend REST API, the View layer is represented by the Express route files. Routes map HTTP verbs and URL patterns to the appropriate controller functions, applying authentication and authorisation middleware as required.

- articleRoutes.js — Exposes GET / (list), GET /categories, GET /:slug (detail), POST / (admin), PUT /:slug (admin), DELETE /:slug (admin).
- authRoutes.js — Exposes POST /register, POST /login, GET /me, POST /forgot-password, POST /reset-password/:token.
- savedArticleRoutes.js — Exposes GET /, POST /, DELETE /:slug — all protected by the auth middleware.

Advantages of Using MVC in This Project

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability: New features (e.g. comments, likes, tags management) can be added by creating new route, controller, and model files without touching existing code.
- Reusability: Controller and service logic (e.g. the news-API fallback chain) can be reused across multiple route handlers.
- Testability: Each layer — especially controllers and models — can be unit-tested in isolation.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflicts.

PROJECT SETUP AND CONFIGURATION

Creating the Project Folder

1. Create a new folder named news-application.
2. Inside it create two sub-folders: client and server.
3. Open the root folder in VS Code.

Client Setup (React + Vite)

Open the client folder in the VS Code terminal and run:

```
npm create vite@latest . -- --template react

# Select framework:
# > React

# Select variant:
# > JavaScript

cd client
npm install
npm run dev           # starts Vite dev server on http://localhost:5173
```

Server Setup (Express + Node)

Open the server folder in a separate terminal and run:

```
npm init -y

# Install dependencies
npm install express mongoose dotenv cors helmet morgan bcryptjs jsonwebtoken
nodemailer slugify

# Install dev dependencies
npm install -D nodemon
```

Create the following files and folders inside server/:

- server.js — Application entry point.
- config/ — db.js (MongoDB connection).
- models/ — Article.js, User.js, SavedArticle.js.
- controllers/ — articleController.js, authController.js, savedArticleController.js.
- routes/ — articleRoutes.js, authRoutes.js, savedArticleRoutes.js.
- middleware/ — authMiddleware.js, errorMiddleware.js.
- services/ — newsApiService.js, mailService.js, tokenService.js.

- data/ — localStore.js, sampleArticles.js (fallback data).
- scripts/ — seed.js (database seeder).
- .env — Environment variables (see below).

Environment Variables

Create a .env file in the server/ directory with the following variables:

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.example.mongodb.net/news-
app
JWT_SECRET=replace-with-a-long-random-secret
ADMIN_ACCESS_CODE=replace-with-private-admin-code
NEWS_API_KEY=your_newsdataio_key_here
CLIENT_ORIGIN=http://localhost:5173

# Optional SMTP settings for password-reset e-mails
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=your-smtp-user
SMTP_PASS=your-smtp-password
SMTP_FROM=noreply@example.com
```

Running the Application

```
# From the monorepo root
npm install          # installs root dev dependencies (concurrently)
npm run dev          # starts both client and server concurrently

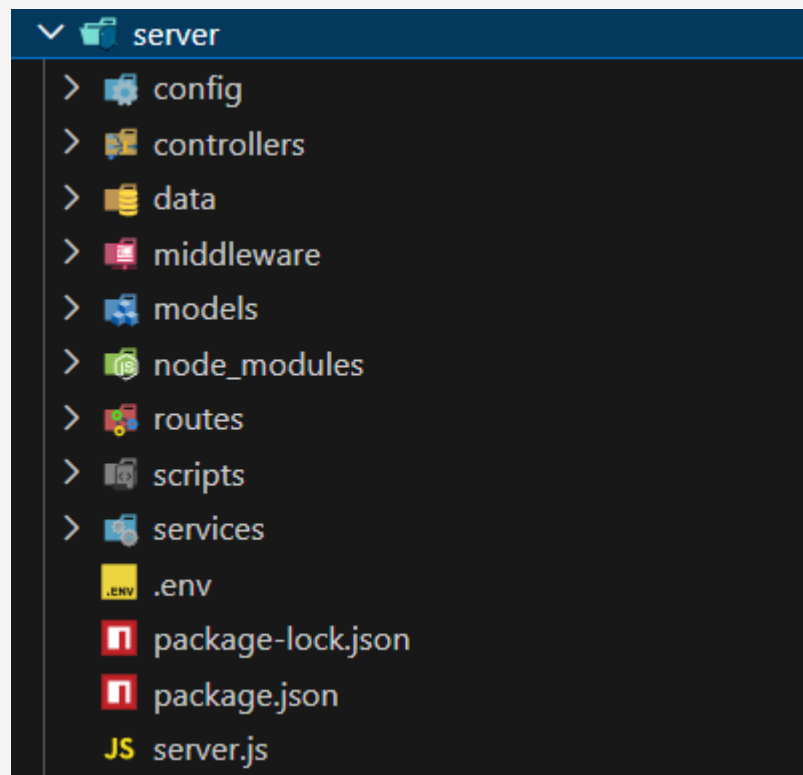
# Seed sample articles into MongoDB
npm run seed
```

BACKEND DEVELOPMENT

Backend Reference Video

[[Backend Reference Video Link](#) — [\[Insert Google Drive Link Here\]](#)]

Backend Structure



Controllers

- `articleController.js` — Handles all article-related business logic: list with pagination/filtering/search, single article by slug, create, update, and delete. Implements a three-tier fallback: NewsData.io API → MongoDB → in-memory sample articles.
- `authController.js` — Handles all authentication operations: register (with role and admin-code validation), login (with constant-time password comparison), get current user, forgot-password (token generation + e-mail delivery), and reset-password (token verification + hash update).
- `savedArticleController.js` — Handles saved-articles CRUD for the authenticated user: get all saved, save new, and remove by slug.

Config

- `db.js` — Contains the Mongoose connection setup. Reads `MONGO_URI` from environment variables. Catches connection failures gracefully and logs a warning so the server continues running in sample-data mode.

Middleware

- `authMiddleware.js` — JWT verification middleware (protect) that extracts and validates the Bearer token, populates `req.user`, and rejects unauthorised requests with 401. A separate `requireAdmin` middleware enforces admin-only routes.
- `errorMiddleware.js` — Global error handler that returns structured JSON error responses with appropriate HTTP status codes, and a 404 handler for unmatched routes.

Models

- `Article.js` — Mongoose schema for news articles with automatic slug generation, text indexes for full-text search, and category/date compound index for efficient filtered queries.
- `User.js` — Mongoose schema for user accounts including `bcrypt` password comparison method, safe JSON serialisation method, and password-reset token fields.
- `SavedArticle.js` — Mongoose schema for bookmarked articles, linking to User by `ObjectId` with a unique compound index on (`user`, `slug`) to prevent duplicate saves.

Routes

- `articleRoutes.js` — Maps HTTP methods and URL patterns to article controller functions. Write operations are gated by `protect` and `requireAdmin` middleware.
- `authRoutes.js` — Maps authentication endpoints to the auth controller.
- `savedArticleRoutes.js` — All routes require the `protect` middleware to ensure only authenticated users can access their saved list.

Services

- `newsApiService.js` — Encapsulates all interaction with the NewsData.io REST API, including request construction, response mapping, and error handling.
- `mailService.js` — Sends password-reset e-mails using Nodemailer. Returns a delivery status flag so the controller can fall back to returning the reset URL in the response if SMTP is not configured.
- `tokenService.js` — Creates signed JWT access tokens with configurable expiry.

API Reference

Method	Endpoint	Access	Description
--------	----------	--------	-------------

GET	/api/health	Public	Service and database health check
GET	/api/articles	Public	List articles (q, category, tag, featured, page, limit)
GET	/api/articles/categories	Public	List available category names
GET	/api/articles/:slug	Public	Get a single article by slug
POST	/api/articles	Admin	Create a new article
PUT	/api/articles/:slug	Admin	Update an article
DELETE	/api/articles/:slug	Admin	Delete an article
POST	/api/auth/register	Public	Register a new user or admin
POST	/api/auth/login	Public	Log in and receive JWT
GET	/api/auth/me	User	Get current authenticated user
POST	/api/auth/forgot-password	Public	Request password-reset link
POST	/api/auth/reset-password/:token	Public	Reset password with token
GET	/api/saved-articles	User	List saved articles for current user
POST	/api/saved-articles	User	Save an article
DELETE	/api/saved-articles/:slug	User	Remove a saved article

DATABASE DEVELOPMENT

1. Configure MongoDB

- Install Mongoose in the server directory:

```
npm install mongoose
```

- Create database connection.
- Create Schemas and Models.

Create Database Connection

Ensure the database is connected before the server starts accepting requests. The connectDB function is called in server.js before app.listen:

```
import mongoose from 'mongoose';

export const connectDB = async () => {
  if (!process.env.MONGO_URI) {
    console.warn('MONGO_URI is not set. Using in-memory sample articles.');
```

```
    return false;
  }
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 5000
    });
    console.log('MongoDB connected');
    return true;
  } catch (error) {
    console.warn('MongoDB unavailable:', error.message);
    return false;
  }
};
```

Create Schemas

Schemas are configured for MongoDB following the entity definitions in the ER diagram. Each schema is shown below.

1) User.js — Schema/model for user accounts

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
```

```

    name: { type: String, trim: true, default: 'Reader' },
    email: { type: String, required: true, unique: true,
lowercase: true },
    passwordHash: { type: String, required: true },
    role: { type: String, enum: ['user','admin'], default: 'user'
},
    gender: { type: String, enum: ['female','male','non-
binary','prefer-not-to-say'] },
    resetPasswordTokenHash: String,
    resetPasswordExpiresAt: Date,
}, { timestamps: true });

export default mongoose.model('User', userSchema);

```

2) Article.js — Schema/model for news articles

```

import mongoose from 'mongoose';

const articleSchema = new mongoose.Schema({
  title: { type: String, required: true, maxlength: 140 },
  slug: { type: String, unique: true, lowercase: true },
  excerpt: { type: String, required: true, maxlength: 280 },
  content: { type: String, required: true },
  author: { type: String, default: 'Newsroom Desk' },
  category: { type: String, enum: categories, default: 'World' },
  imageUrl: { type: String, required: true },
  tags: { type: [String], default: [] },
  featured: { type: Boolean, default: false },
  readTime: { type: Number, default: 4 },
  publishedAt: { type: Date, default: Date.now },
}, { timestamps: true });

articleSchema.pre('validate', function(next) {
  if (!this.slug && this.title)
    this.slug = slugify(this.title, { lower: true, strict: true });
  next();
});

export default mongoose.model('Article', articleSchema);

```

3) SavedArticle.js — Schema/model for user saved articles

```

import mongoose from 'mongoose';

const savedArticleSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true, index: true },
  slug: { type: String, required: true },
  title: String,
  excerpt: String,

```

```
category: String,
imageUrl: String,
tags:     { type: [String], default: [] },
readTime: Number,
publishedAt: Date,
}, { timestamps: true });

savedArticleSchema.index({ user: 1, slug: 1 }, { unique: true });

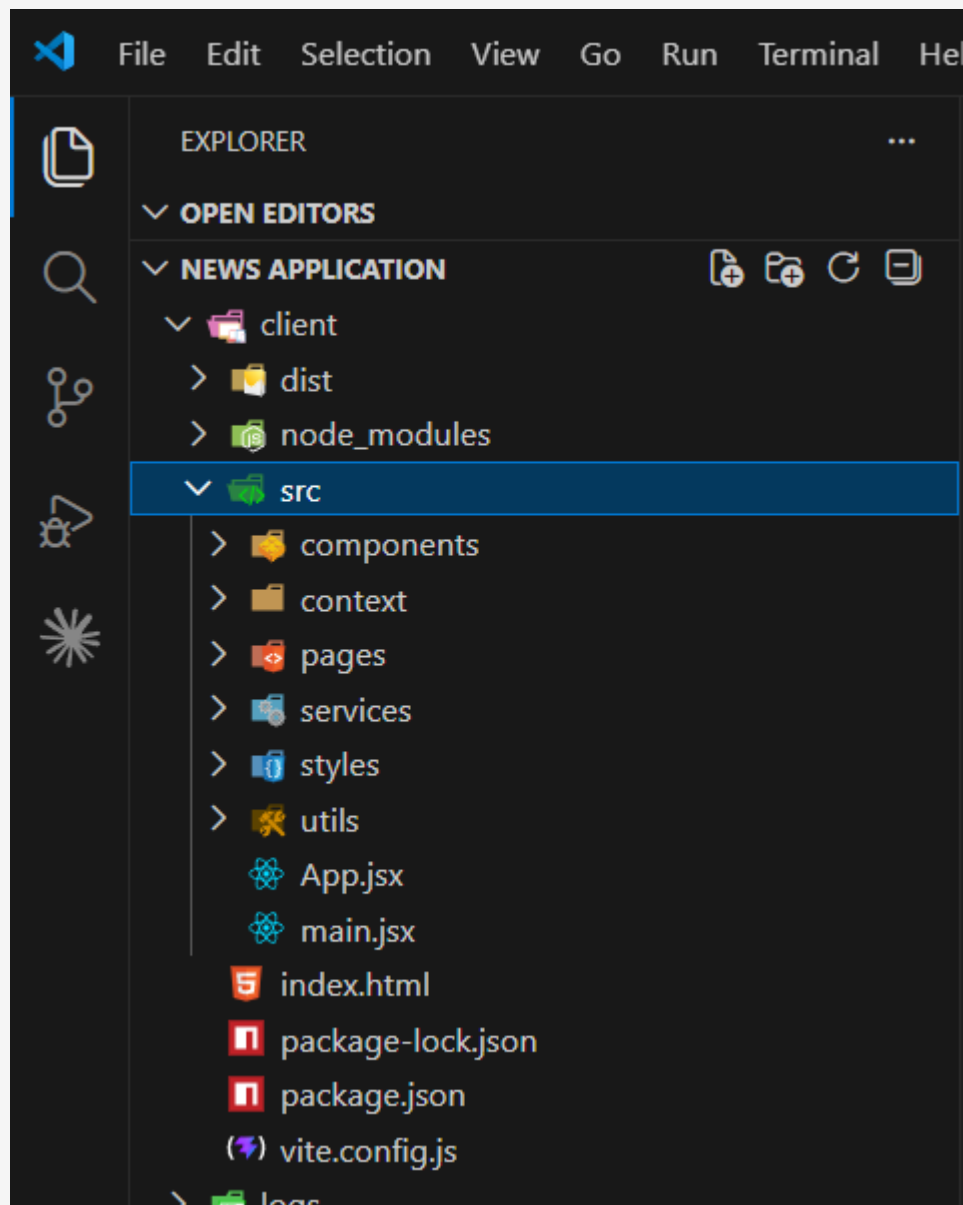
export default mongoose.model('SavedArticle', savedArticleSchema);
```

FRONT-END DEVELOPMENT

Frontend Reference Video

[[Frontend Reference Video Link](#) — [Insert Google Drive Link Here]]

Frontend Structure



Pages (src/pages)

- `HomePage.jsx` — Main landing page. Fetches articles from the API, renders featured cards and category-filtered lists, and integrates `SearchBar` and `CategoryTabs`.
- `ArticlePage.jsx` — Full article detail page. Loads article by `:slug`, renders cover image, full content, metadata, and a `Save` button for authenticated users.
- `AuthPage.jsx` — Combined Login / Register page with tabs. Handles both user and admin sign-in flows, including the admin access code field.
- `ForgotPasswordPage.jsx` — Displays a form for entering an e-mail address to trigger password-reset link delivery.
- `ResetPasswordPage.jsx` — Displays a form to enter a new password, consuming the reset token from the URL.
- `SavedPage.jsx` — Protected page listing all articles saved by the current user, with a remove option for each.
- `ProfilePage.jsx` — Protected page displaying the current user's profile information.
- `PublishPage.jsx` — Admin-only protected page with a form to create new articles (title, slug preview, excerpt, content, category, tags, `imageUrl`, `readTime`, featured flag).

Components (src/components)

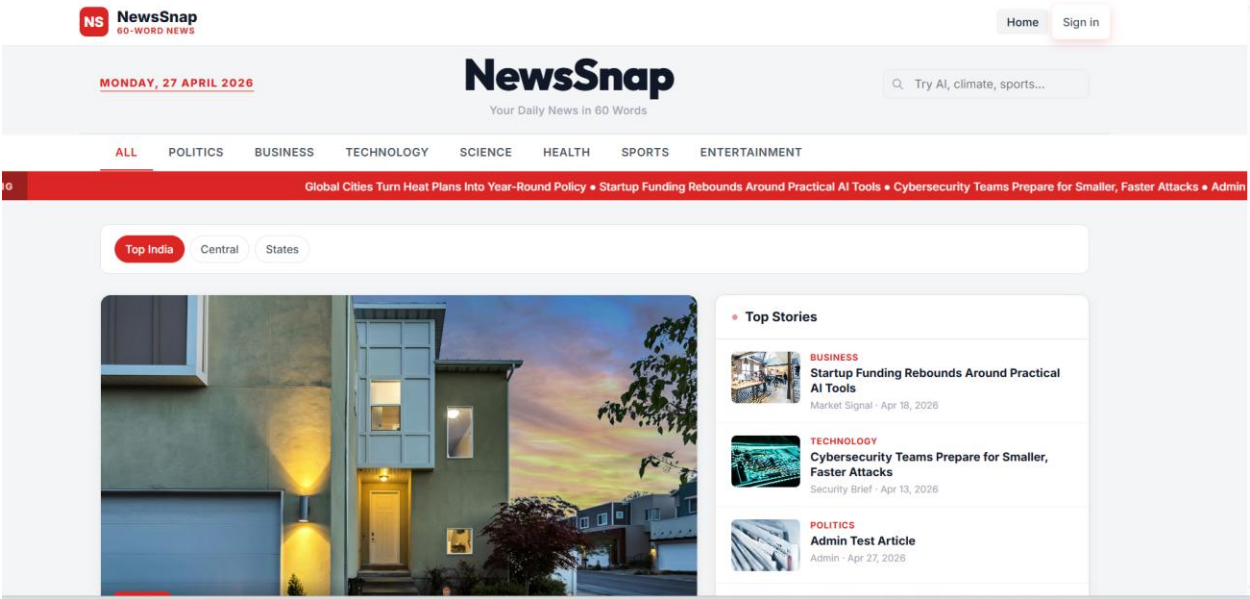
- `Header.jsx` — Top navigation bar with the brand logo, links to Home, Saved, Profile, and Publish (admin only), and Login / Logout actions.
- `Footer.jsx` — Site-wide footer with copyright and contact information.
- `ArticleCard.jsx` — Reusable card component that displays article thumbnail, category badge, title, excerpt, author, and read time.
- `CategoryTabs.jsx` — Horizontal scrollable row of category filter buttons.
- `SearchBar.jsx` — Debounced text input that emits the current search query to the parent page.
- `AvatarBadge.jsx` — Gender-aware avatar component used in the profile header.
- `LoadingState.jsx` — Skeleton / spinner displayed while data is being fetched.
- `ProtectedRoute.jsx` — Higher-order route component that redirects unauthenticated users to `/auth` and optionally restricts admin-only routes.

Context and Services (src/context, src/services)

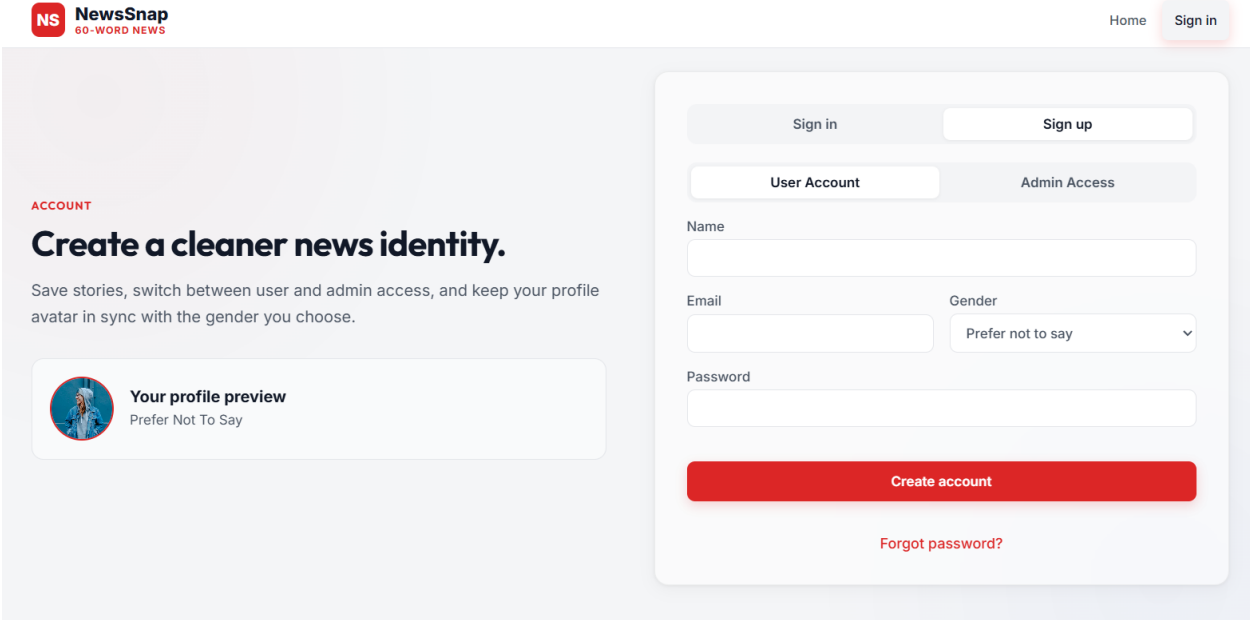
- `AuthContext.jsx` — React context that stores the current user and JWT token, exposes login/logout helpers, and persists state to `localStorage`.
- `api.js` — Axios instance pre-configured with the base URL and a request interceptor that attaches the JWT Authorization header from the `AuthContext`.

OUTPUT SCREENSHOTS

Landing Page (Home)



Register Page



Login Page

ACCOUNT

Welcome back to the India desk.

Save stories, switch between user and admin access, and keep your profile avatar in sync with the gender you choose.



Your profile preview
Female

Sign in

Sign up

User Account

Admin Access

Email

sandhyaranikorne@gmail.com

Gender

Female

Password

.....

Admin access code

.....

Sign in

[Forgot password?](#)

Article Detail Page

[— Back to newsroom](#)

HEALTH

Hospitals Test Ambient Notes to Reduce Clinician Burnout

Pilot programs are using secure voice-to-note systems to cut documentation time while keeping doctors in control of final records.

BY ELENA BROOKS · APR 17, 2026 · 4 MIN READ

[Save for later](#)



Health systems are experimenting with tools that draft visit notes from patient conversations after receiving consent. Clinicians review, edit, and approve the final record before it enters the chart. Early pilots report shorter after-hours documentation sessions, though administrators are moving carefully around privacy, accuracy, and liability.

[health](#) [ai](#) [hospitals](#)

[Read full story at Health Systems Review ↗](#)

LATEST HEADLINES

Stay updated with the most recent stories from our editors.

[View all news —](#)

Saved Articles Page

NS

NewsSnap

60-WORD NEWS

Home

Saved

Profile

Sign out

WATCH LATER

Your saved stories



SPORTS · APR 26, 2026 · 1 MIN READ

Torino vs Inter Milan, Serie A 2025-26 Free Live Streaming Online

The Nerazzurri currently sit nine points clear at the top of the table and could be crowned champions...

Top India

Sports

Latestly

Save

Remove from list



CENTRAL · APR 26, 2026 · 1 MIN READ

Raghuvanshi, KKR fume after rare obstruction dismissal

The young batter became only the fourth Indian player in IPL history to be given out obstructing th...

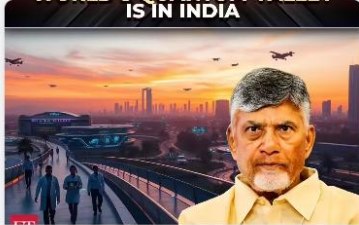
Central

Sports

Hindustan Times

Save

Remove from list



ANDHRA PRADESH · APR 25, 2026 · 1 MIN READ

ET Awards 2025 | Amravati is world's 'Quantum Valley': Chandrababu Naidu, CM, Andhra Pradesh

The ET Awards for Corporate Excellence serve as a benchmark for India Inc, bringing together top...

Andhra Pradesh

Business

The Economic Times

Save

Remove from list

Publish Page (Admin)

EDITOR

Publish an admin story

Title

Excerpt

Story

Author

Category

Politics

Read time (min)

4

Image URL

<https://images.unsplash.com/photo-1504711434969-e33886168f5c?auto=format&fit=crop&w=1200&q=80>

Source name

Source URL

Tags

climate, policy, cities

☐ Feature this story on homepage

Publish story

[Forgot Password Page](#)

ACCOUNT

Reset your password

Registered email

Send reset email



NewsSnap — Your daily news in 60 words. Short, concise, and always up to date.
© 2026 NewsSnap. All rights reserved.

[Home](#) [Sign In](#)

Reset Password Page

ACCOUNT

Choose a new password

New password

Reset password

Profile Page

PROFILE

sandhya rani

✓ Profile updated successfully



sandhya rani
ADMIN

EMAIL
sandy2211@gmail.com

GENDER
Female

ROLE
Admin

SAVED
0 stories

Edit Profile

Actions

Saved Stories

Publish Story

Change Password

Delete Account

Sign Out

DEMO VIDEO LINK

Demo Video Link:

https://drive.google.com/drive/folders/1q3oonPN6n268jLezYKT6c_eT0aT0LrRy?usp=drive_link

CODE DRIVE LINK

Code Drive Link:

https://drive.google.com/drive/folders/1VLKNTePnrqItDtxhB_P6sjmphlbw6J6R?usp=drive_link